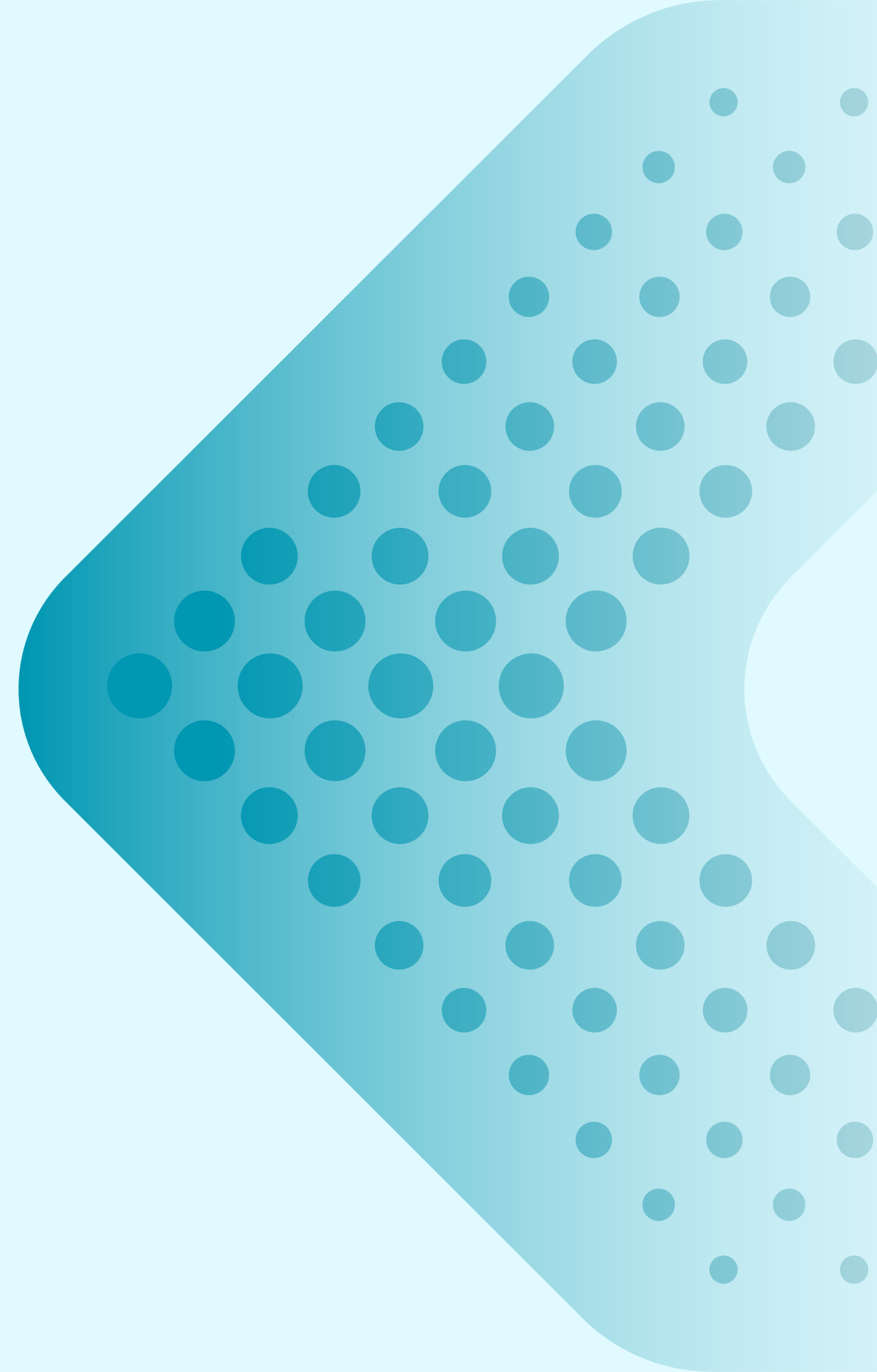


ENSEMBLE MODELS

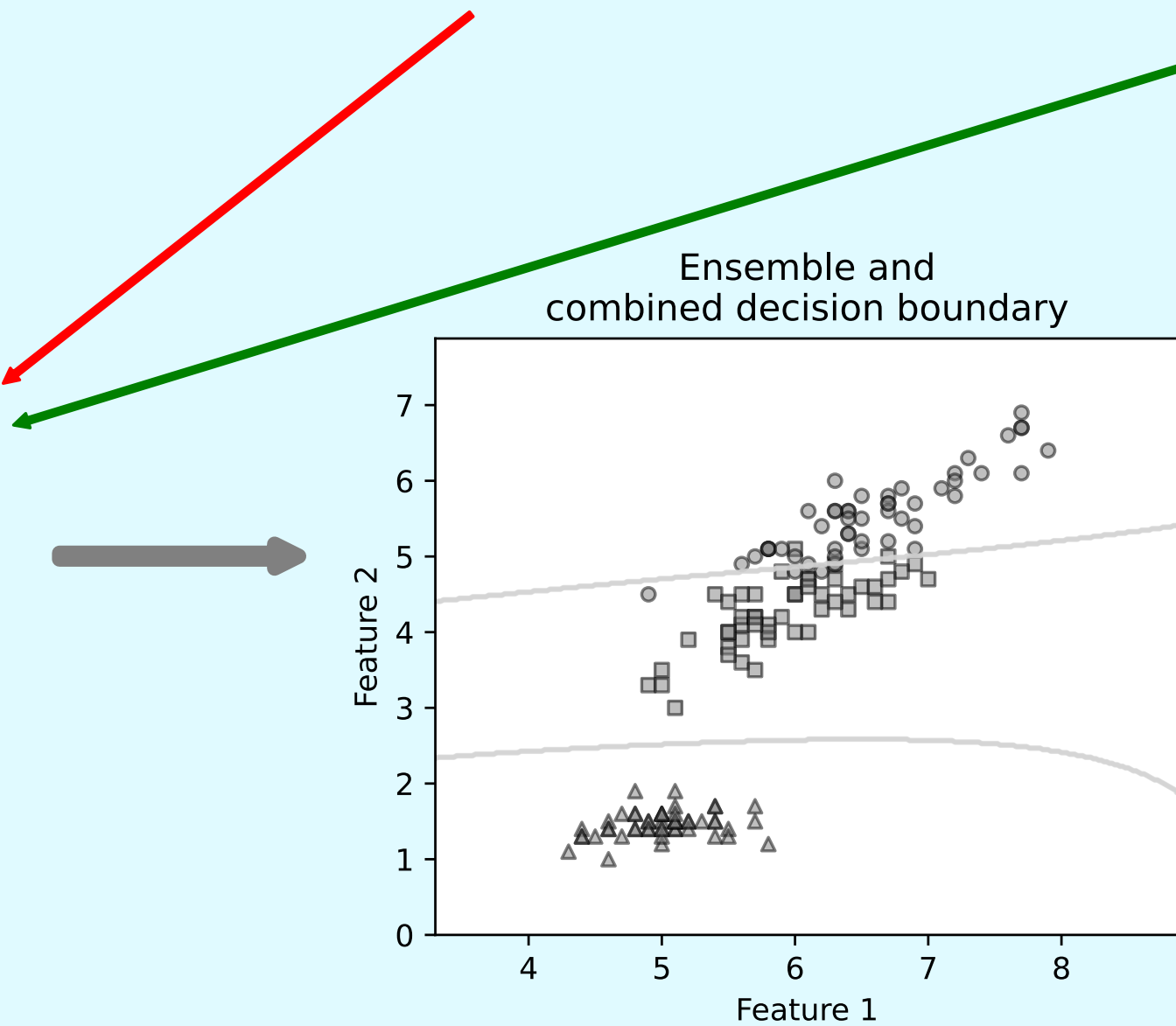
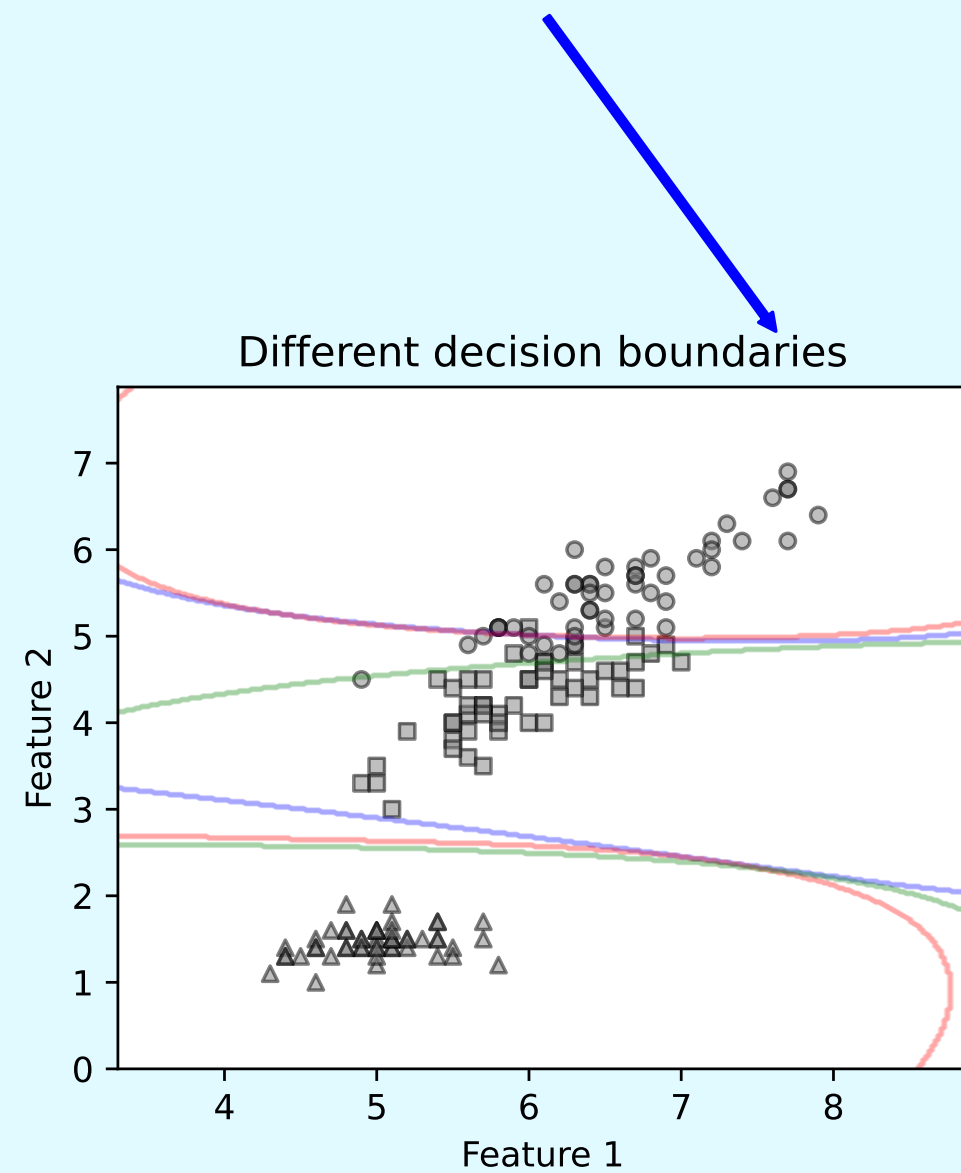
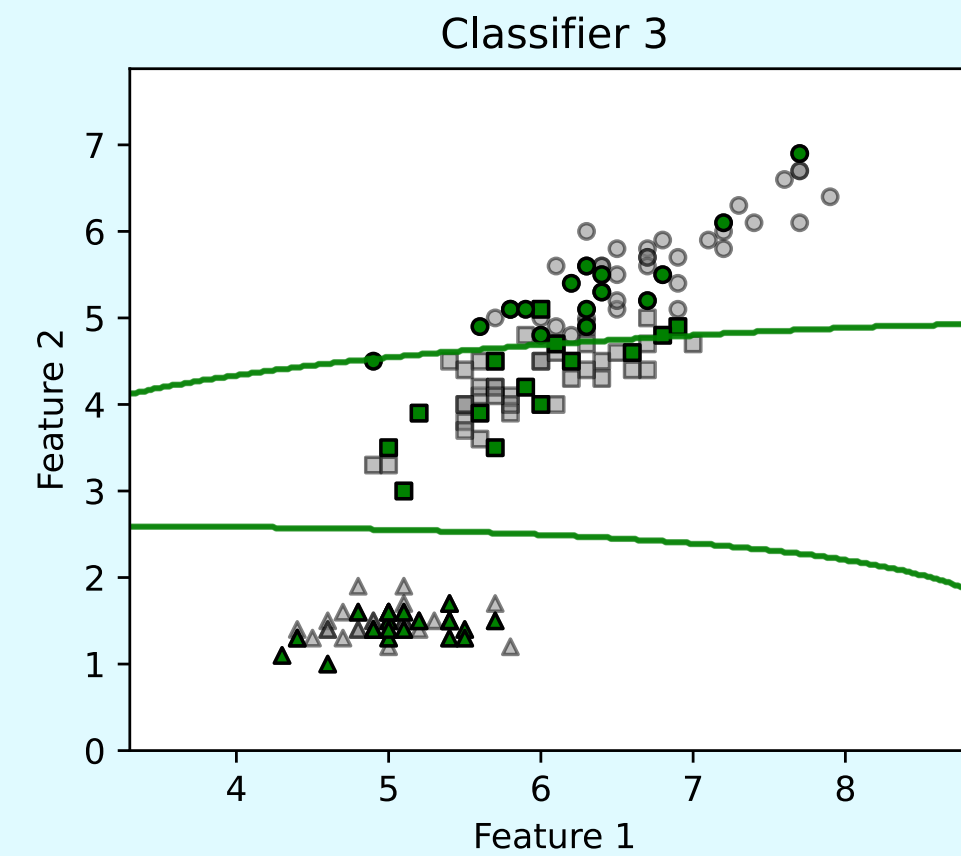
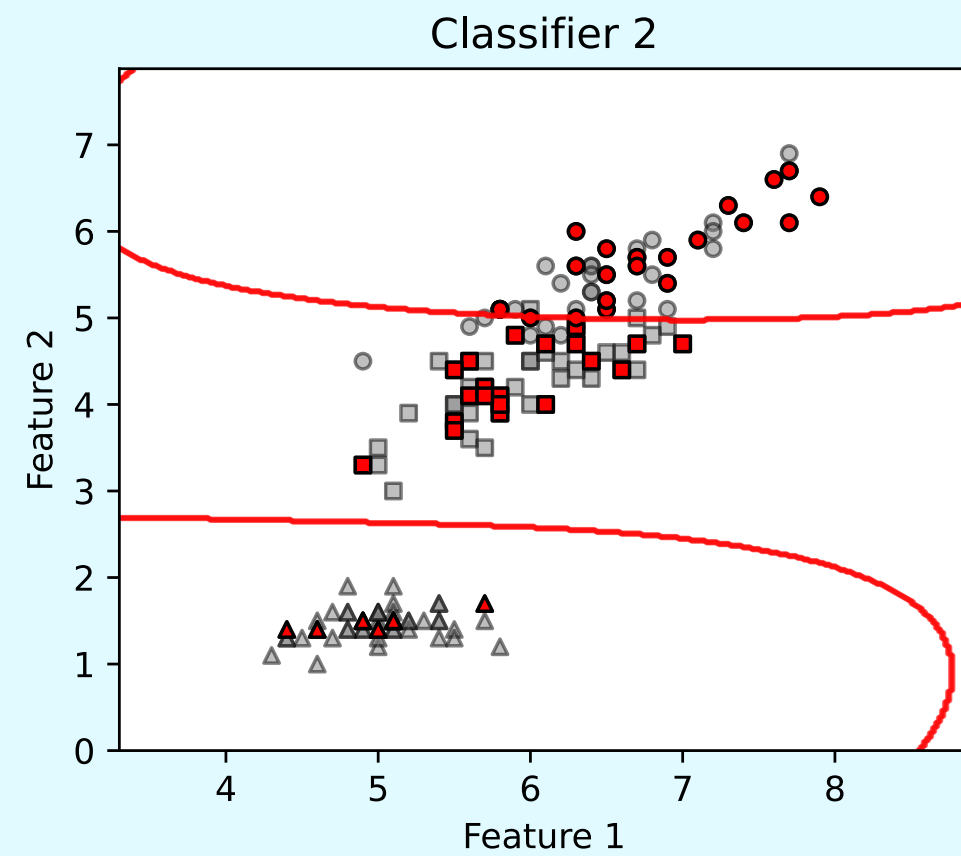
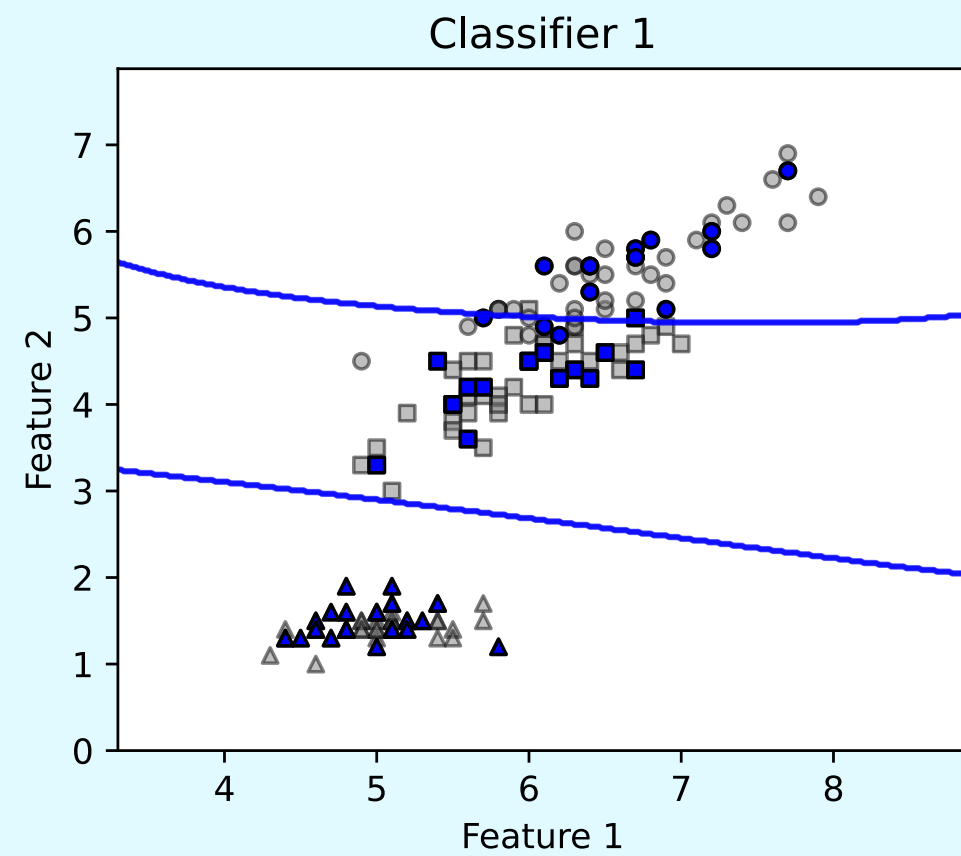
BY MAHA OZAIR, FARAZ HUSSAIN



WHAT IS ENSEMBLE LEARNING?

Ensemble learning is a method where we use many small models instead of just one. Each of these models may not be very strong on its own, but when we put their results together, we get a better and more accurate answer.

The idea is that many weak learners can come together to form a strong learner.



- △ Class 0
- Class 1
- Class 2
- ★ Classifier 1 training subset
- ★ Classifier 1 training subset
- ★ Classifier 1 training subset
- Classifier 1 boundary
- Classifier 2 boundary
- Classifier 3 boundary
- Ensemble boundary

PROBLEM

- Limited Learning Capacity: One model may only capture certain types of patterns, missing relationships that require different modeling approaches.
- High Sensitivity to Data: Some models produce unstable predictions if the training data changes slightly, making them unreliable.
- Risk of Overfitting or Underfitting: A single model can either be too simple to learn the data or too complex and memorize noise instead of patterns.
- Weak Performance on Complex or Imbalanced Data: Single models often struggle when classes are imbalanced or when the data has nonlinear structure.
- No Model Works Best for Every Problem: Different algorithms excel at different aspects of a dataset, so relying on only one can limit overall performance.

Ensemble Learning

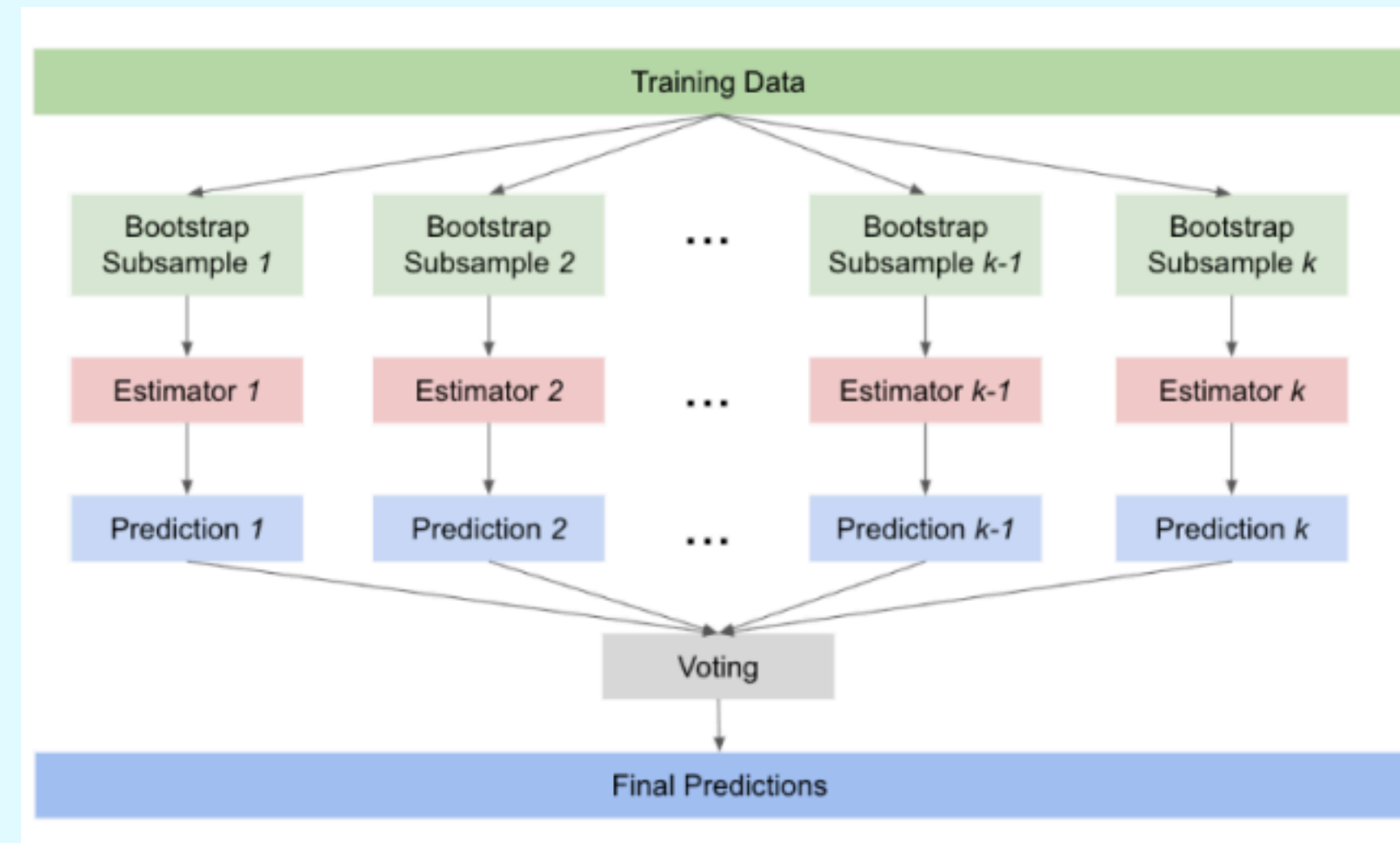
```
graph TD; A[Ensemble Learning] --> B[Bagging]; A --> C[Boosting]; A --> D[Stacking];
```

Bagging

Boosting

Stacking

BAGGING

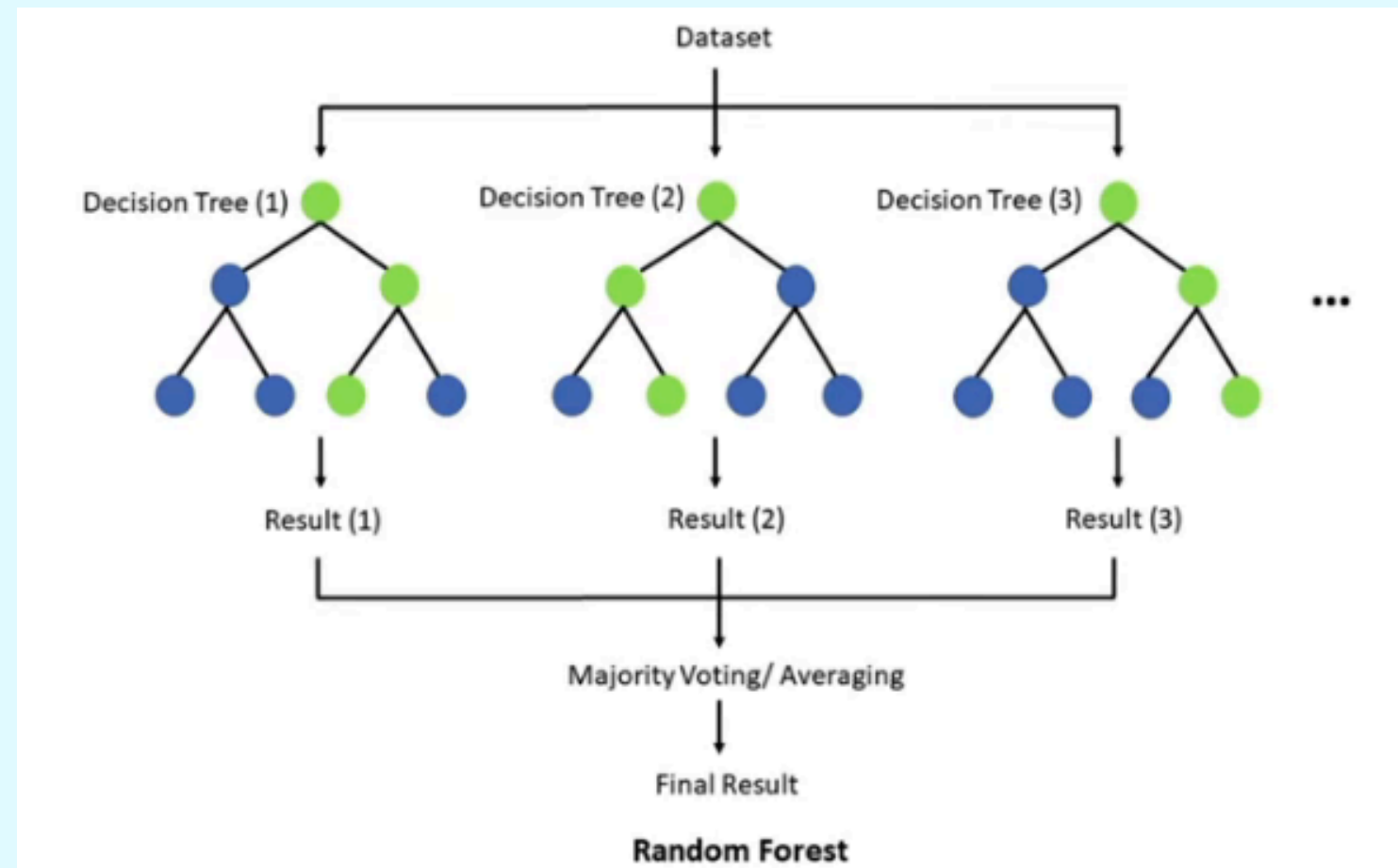


During bagging each model is trained on a slightly different version of the dataset, so they learn different patterns. When their predictions are combined, the result is more stable and less likely to overfit than relying on just one model.

BAGGING ALGORITHM

- **Bootstrap Sampling:** Divides the original training data into 'N' subsets and randomly selects a subset with replacement in some rows from other subsets. This step ensures that the base models are trained on diverse subsets of the data and there is no class imbalance.
- **Base Model Training:** For each bootstrapped sample we train a base model independently on that subset of data. These weak models are trained in parallel to increase computational efficiency and reduce time consumption. We can use different base learners i.e. different ML models as base learners to bring variety and robustness.
- **Prediction Aggregation:** To make a prediction on testing data combine the predictions of all base models. For classification tasks it can include majority voting or weighted majority while for regression it involves averaging the predictions.
- **Final Prediction:** After aggregating the predictions from all the base models, Bagging produces a final prediction for each instance.

RANDOM FOREST



- BAGGING TRAINS MULTIPLE MODELS (LIKE DECISION TREES) ON DIFFERENT SAMPLED VERSIONS OF THE DATASET, ALLOWING EACH MODEL TO LEARN SLIGHTLY DIFFERENT PATTERNS AND NOISE.
- DECISION TREES ARE IDEAL FOR BAGGING BECAUSE THEY CAN GROW DEEP AND CAPTURE COMPLEX INTERACTIONS, BUT THEY ARE ALSO HIGHLY VARIABLE AND PRONE TO OVERFITTING, MEANING THEY BENEFIT A LOT FROM AGGREGATION.
- RANDOM FOREST APPLIES BAGGING TO MANY DECISION TREES AND COMBINES THEIR PREDICTIONS, WHICH REDUCES VARIANCE WHILE KEEPING LOW BIAS, RESULTING IN MORE STABLE AND ACCURATE PREDICTIONS THAN A SINGLE TREE.

BAGGING EXAMPLE

Original dataset:

- I love NLP (positive)
- I enjoy eating pizza (positive)
- The sky is blue today (nueutral)

Bootsrapped Dataset 1:

- I enjoy eating pizza
- I love NLP
- I love NLP

Bootstrapped Dataset 2:

- The sky is blue today
- I enjoy eating pizza
- I enjoy eating pizza

Bootstrapped Dataset 3:

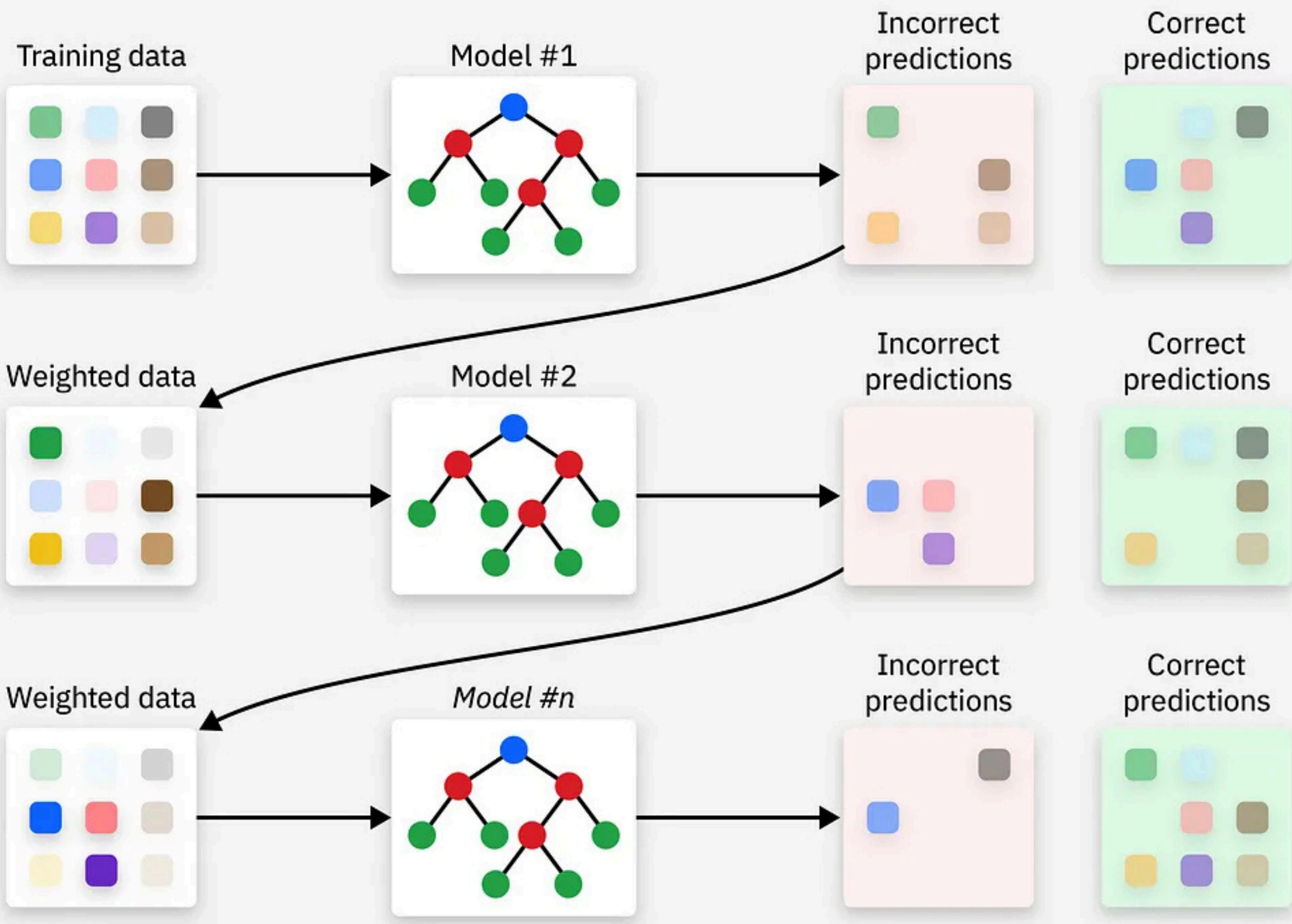
- The sky is blue today
- I love NLP
- The sky is blue today

Train a model on each dataset and see predictions for test sentence : "I think NLP is amazing"

- Model 1 predicts: positive
- Model 2 predicts: positive
- Model 3 predicts: nueutral
- Final bagged prediction (majority vote): positive

BOOSTING

- Boosting is an ensemble technique that combines multiple weak learners to create a strong learner.
- Weak models are trained in series such that each next model tries to correct errors of the previous model until the entire training dataset is predicted correctly.
- One of the most well-known boosting algorithms is AdaBoost (Adaptive Boosting).



BOOSTING ALGORITHM

- Initialize Model Weights: Begin with a single weak learner and assign equal weights to all training examples.
- Train Weak Learner: Train weak learners on these dataset.
- Sequential Learning: Boosting works by training models sequentially where each model focuses on correcting the errors of its predecessor. Boosting typically uses a single type of weak learner like decision trees.
- Weight Adjustment: Boosting assigns weights to training datapoints. Misclassified examples receive higher weights in the next iteration so that next models pay more attention to them.

BOOSTING IN NLP: STEP-BY-STEP OVERVIEW

1. Start with a Weak NLP Model

Train a simple classifier on text data (e.g., bag-of-words + decision stump or basic Naive Bayes).
It captures only obvious patterns, so it misclassifies many reviews.

2. Increase Weights on Misclassified Text Samples

The training algorithm assigns higher importance to the samples the first model got wrong.
These “hard” examples now influence the next model more strongly.

3. Train the Next Model Focused on the Hard Cases

The second model learns patterns the first model missed:

- Slang
- Negations
- Multi-word expressions
- Domain-specific terms

Its job is to correct the earlier mistakes.

4. Repeat the Cycle

Each new model trains on reweighted data, specializing further:

- Subtle sentiment cues
- Sarcasm and tone shifts
- Long or complex sentence structures
- Rare vocabulary

Errors shrink with each iteration.

5. Combine All Models into One Strong Classifier

Boosting assigns a weight to each model based on accuracy.

Final prediction = weighted vote of all weak learners.

6. Result

A high-performing NLP model that captures simple patterns, tricky edge cases, and subtle linguistic cues far better than any single classifier.

STACKING

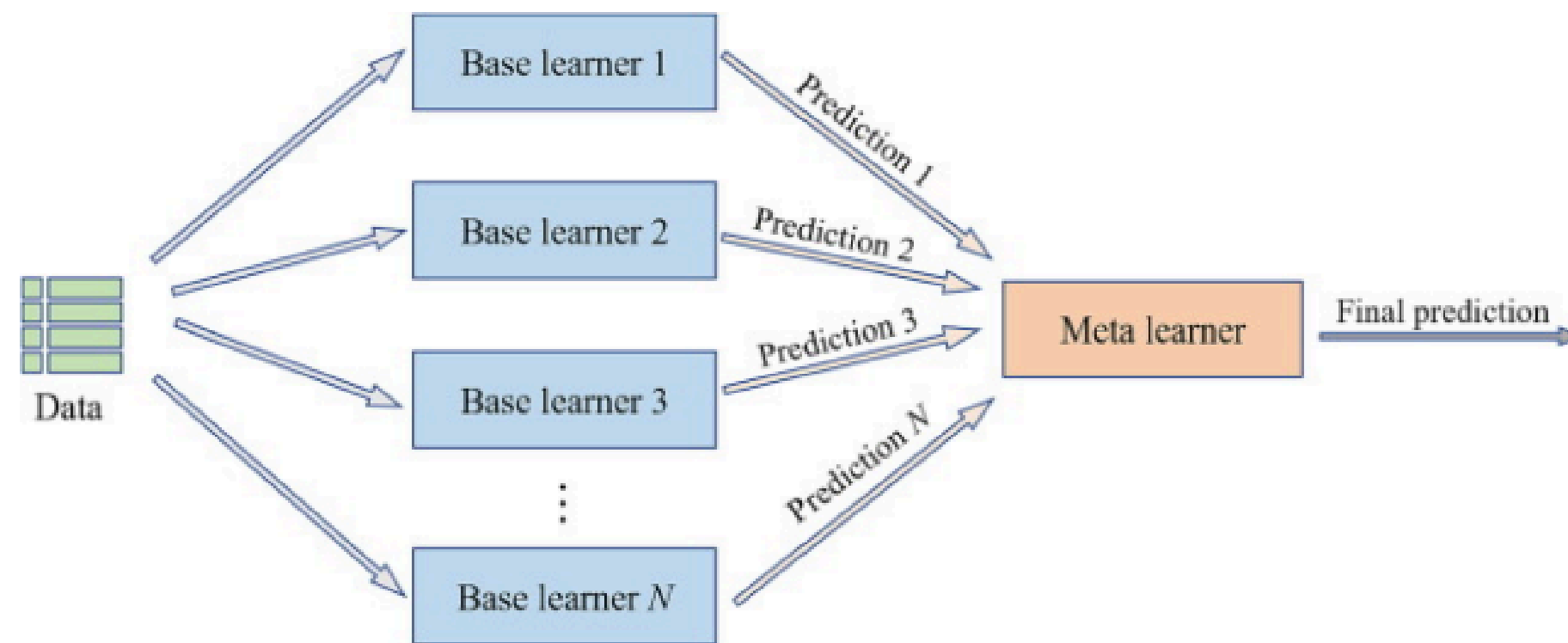


Figure 2. Overview of the stacking architecture, where base models provide predictions used as features by a meta-learner.

During stacking the outputs of the base learner models are used as an input to the meta learner layer. The meta-learner learns how to best combine the strengths of the base models, reducing their individual biases and improving overall performance.

STACKING ALGORITHM

Input: Training dataset D

1. Select k base learning models (Model1 ... Model k).
2. Train each base learner on the full dataset D .
3. Use cross-validation to generate predictions for each model, creating a new dataset D' where features = predictions from the base models.
(its simulating how it would predict based on unseen data)
4. Train a meta-learner on D' .
5. For a new unseen input x :
 - a. Obtain predictions from each base learner and create a vector v
 - b. Feed v to the meta-learner

Output: Final prediction from the meta-learner.

STACKING EXAMPLE

step 1:

Original Labelled Dataset

I love NLP → positive

I enjoy eating pizza → positive

The sky is blue today → neutral

step 2

Stack Predictions

model 1: [positive, neutral
positive]

model 2: [neutral, neutral
positive]

model 3: [positive, positive,
positive]

These predictions become
the features for the meta-
learner.

step 3:

Train the Meta-Learner

Meta-learner

input: Base model predictions
on training data (meta-
features)

target: Original labels
The meta-learner learns how
to combine base model
outputs to improve overall
prediction.

EXAMPLE CONTINUED

For testing

test sentence: "I think NLP is amazing"

Base learners predict individually:

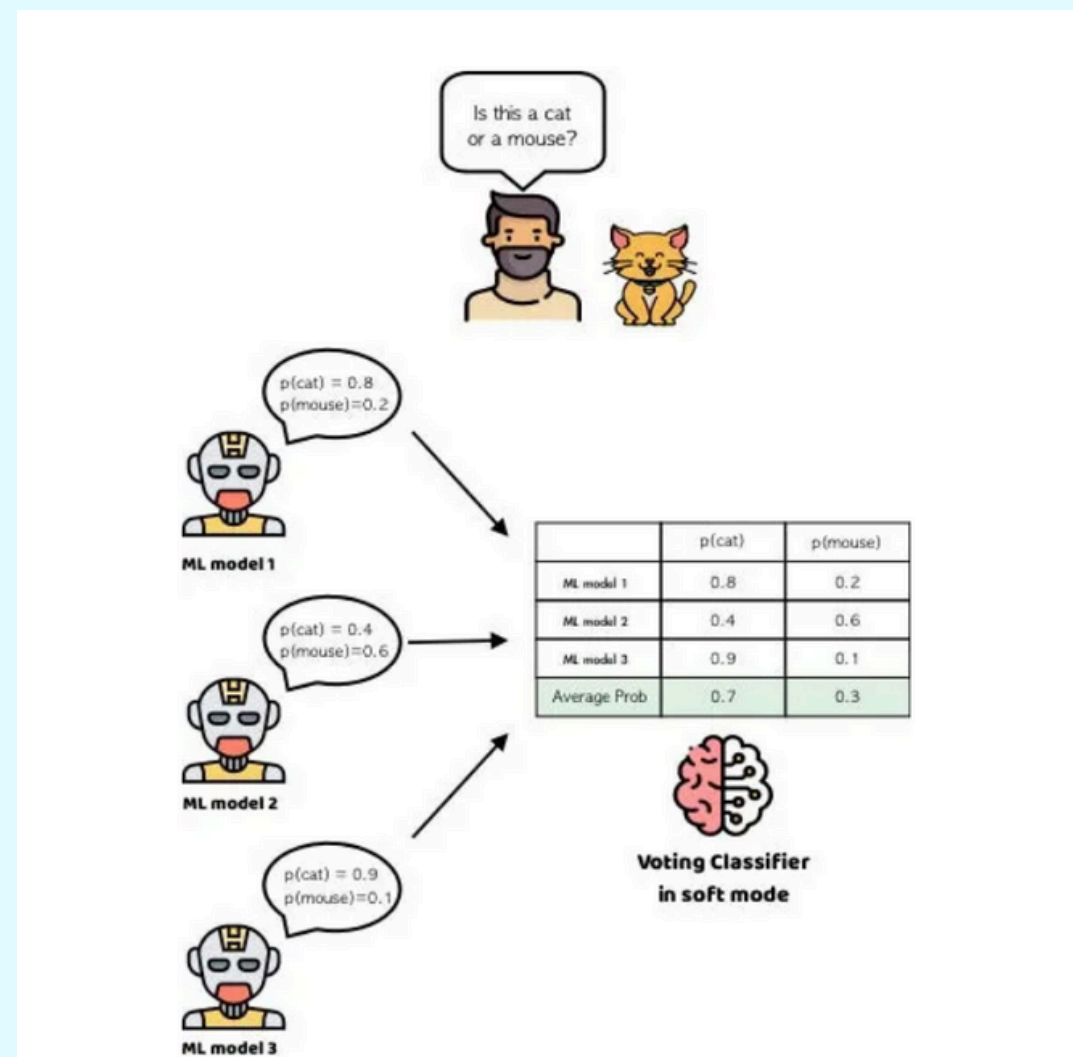
Model 1 prediction: positive

Model 2 prediction: neutral

Model 3 prediction: positive

1. Create meta-feature vector from these predictions: [positive, neutral, positive]
2. Feed meta-feature vector into trained meta-learner this would create a final stacked prediction: "positive"

VOTING ENSEMBLES

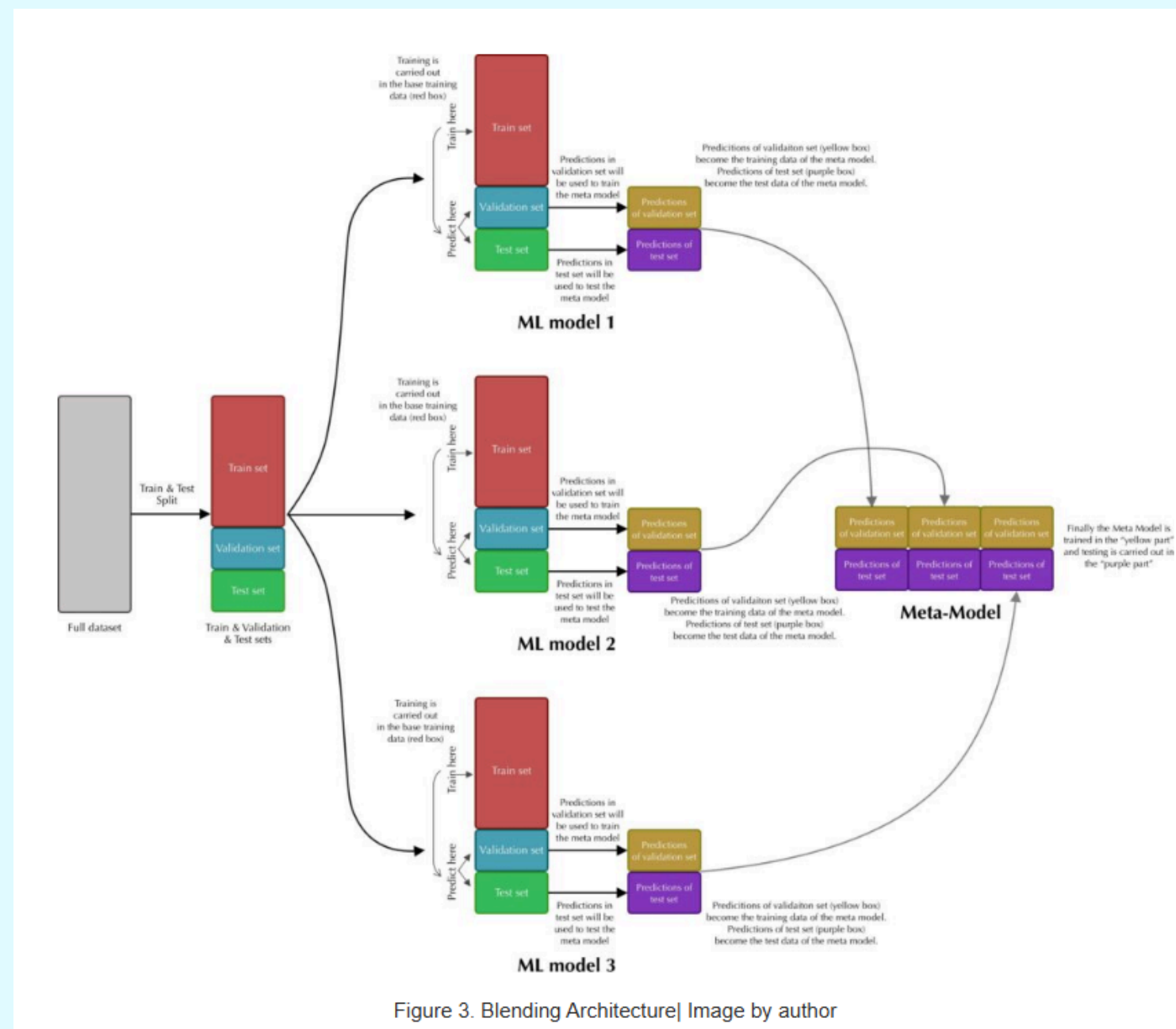


- VOTING ENSEMBLES COMBINE PREDICTIONS FROM MULTIPLE MODELS USING MAJORITY OR WEIGHTED VOTING.
- MAJORITY VOTING (HARD VOTING): EACH MODEL MAKES A PREDICTION, AND THE CLASS WITH THE MOST VOTES BECOMES THE FINAL PREDICTION.
 - WEIGHTED VOTING (SOFT VOTING): SIMILAR TO MAJORITY VOTING, BUT SOME MODELS' VOTES COUNT MORE THAN OTHERS BASED ON THEIR PERFORMANCE OR CONFIDENCE. IS USEFUL WHEN YOU HAVE MULTIPLE WELL PERFORMING MODELS
 - IS GREAT WHEN YOU NEED TO IMPLEMENT SOMETHING LESS COMPLEX

BLENDING ENSEMBLES

COMBINES PREDICTIONS FROM MULTIPLE BASE MODELS BY TRAINING A HOLDOUT DATASET TO FEED INTO A META-LEARNER, WHICH MAKES THE FINAL PREDICTION.

- META-LEARNER LEARNS HOW TO COMBINE BASE MODEL PREDICTIONS TO IMPROVE OVERALL ACCURACY.
- USEFUL WHEN: YOU WANT HIGHER PREDICTIVE PERFORMANCE THAN VOTING BUT WITHOUT THE FULL COMPLEXITY OF CROSS-VALIDATION STACKING.
- EASIER TO IMPLEMENT THAN FULL STACKING BECAUSE IT USES A SINGLE HOLDOUT SET INSTEAD OF MULTIPLE FOLDS.



ENSEMBLE LEARNING TECHNIQUES

Random Forest	Bagging	Random forest constructs multiple decision trees on bootstrapped subsets of the data and aggregates their predictions for final output, reducing overfitting and variance.
Random Subspace Method	Bagging	Trains models on random subsets of input features to enhance diversity and improve generalization while reducing overfitting.
Gradient Boosting Machines (GBM)	Boosting	Gradient Boosting Machines sequentially builds decision trees, with each tree correcting errors of the previous ones, enhancing predictive accuracy iteratively.
Extreme Gradient Boosting (XGBoost)	Boosting	XGBoost do optimizations like tree pruning, regularization and parallel processing for robust and efficient predictive models.
AdaBoost (Adaptive Boosting)	Boosting	AdaBoost focuses on challenging examples by assigning weights to data points. Combines weak classifiers with weighted voting for final predictions.
CatBoost	Boosting	CatBoost specialize in handling categorical features natively without extensive preprocessing with high predictive accuracy and automatic overfitting handling.

ADVANTAGES

BAGGING ADVANTAGES:

- VARIANCE REDUCTION: REDUCES OVERFITTING BY AVERAGING PREDICTIONS FROM MULTIPLE LEARNERS.
- ROBUSTNESS TO NOISY DATA: DILUTES THE EFFECT OF OUTLIERS THROUGH BOOTSTRAP SAMPLING.
- PARALLELIZABLE TRAINING: BASE LEARNERS ARE INDEPENDENT AND CAN BE TRAINED IN PARALLEL.
- IT ALSO PERFORMS WELL ON HIGH-DIMENSIONAL DATA.

BOOSTING ADVANTAGES:

- BIAS REDUCTION: SEQUENTIALLY CORRECTS ERRORS TO REDUCE BIAS.
- IMPROVED ACCURACY: LEARNS FROM MISTAKES TO BOOST OVERALL PREDICTIVE PERFORMANCE.
- HANDLES IMBALANCED DATA: FOCUSES MORE ON HARD-TO-CLASSIFY INSTANCES.
- ADAPTIVITY: LATER LEARNERS ADAPT TO THE ERRORS OF EARLIER LEARNERS.
- REGULARIZATION OPTIONS: HYPERPARAMETERS AND EARLY STOPPING HELP PREVENT OVERFITTING.

STACKING ADVANTAGES:

- MODEL FLEXIBILITY: COMBINES DIFFERENT TYPES OF BASE LEARNERS EFFECTIVELY.
- HIGHER PREDICTIVE PERFORMANCE: META-LEARNER LEARNS HOW TO OPTIMALLY COMBINE PREDICTIONS.
- BIAS-VARIANCE TRADEOFF: CAN REDUCE BOTH BIAS AND VARIANCE DEPENDING ON SETUP.
- IT CAN EFFECTIVELY COMBINE THE STRENGTHS OF BASE MODELS THUS ENABLING THE POSSIBILITY OF OUTPERFORMING EVEN THE MOST ACCURATE BASE MODEL.

LIMITATIONS / TRADE-OFFS

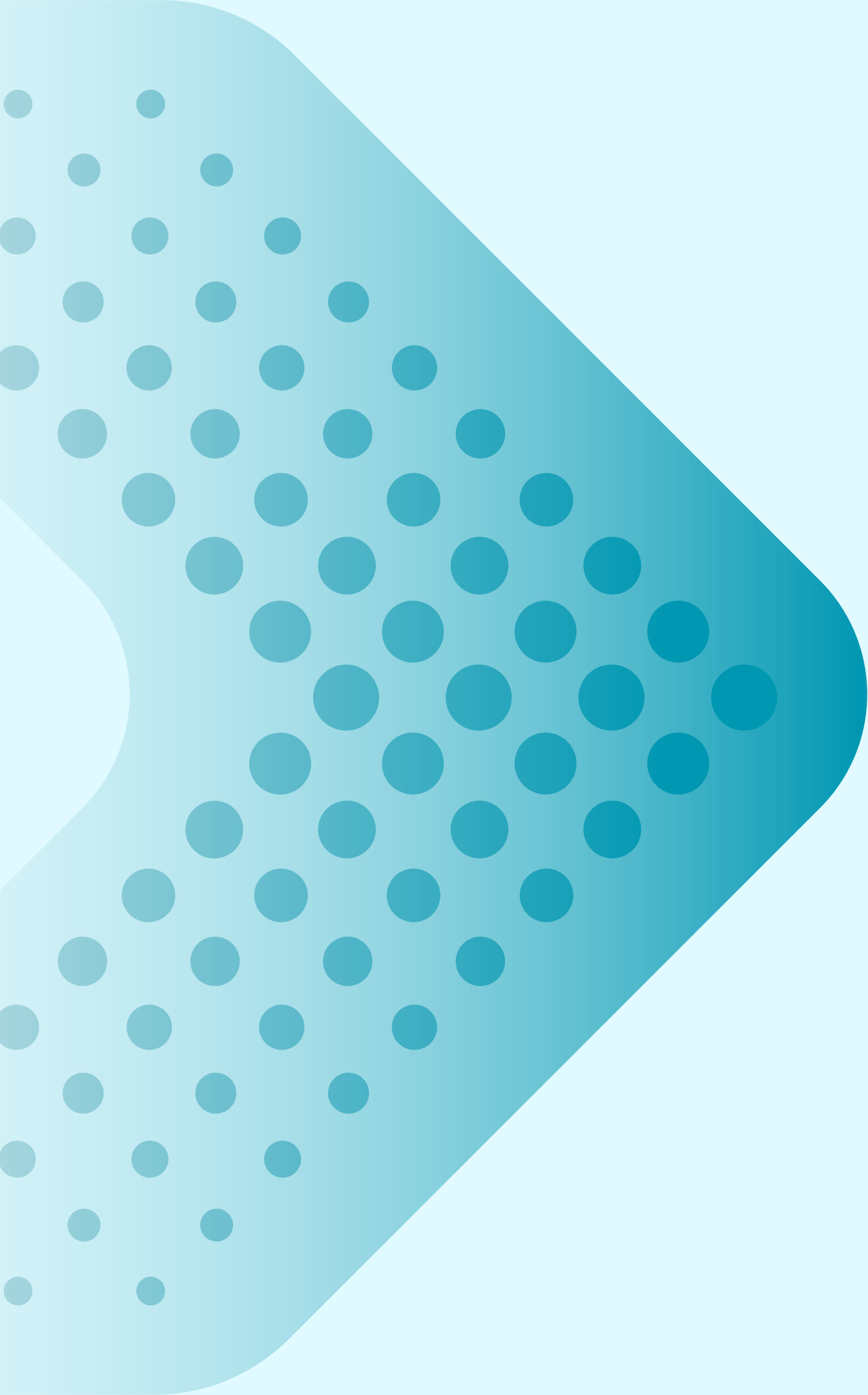
- **COMPUTATIONAL COMPLEXITY:** TRAINING MULTIPLE OR SEQUENTIAL MODELS CAN BE EXPENSIVE.
- **RISK OF OVERFITTING:** BOOSTING AND STACKING CAN OVERFIT IF NOT CONFIGURED CAREFULLY.
- **IMPLEMENTATION COMPLEXITY:** DESIGNING A GOOD ENSEMBLE REQUIRES CAREFUL PLANNING.
- **INTERPRETABILITY:** ENSEMBLES ARE OFTEN HARDER TO INTERPRET THAN SINGLE MODELS.
- **DATA REQUIREMENTS:** STACKING NEEDS ENOUGH DATA TO GENERATE RELIABLE META-FEATURES.

KEY ADVANCEMENTS

- **MIXTURE-OF-EXPERTS (MOE):** THIS ARCHITECTURE, COMMON IN LARGE-SCALE MODELS LIKE GOOGLE'S SWITCH TRANSFORMER, DYNAMICALLY ROUTES DIFFERENT INPUTS TO SPECIALIZED "EXPERT" SUB-MODELS, GATING NETWORK , WHICH DETERMINES THE WEIGHTS ASSIGNED TO EACH EXPERT FOR A GIVEN INPUT X SIGNIFICANTLY IMPROVING SCALABILITY AND EFFICIENCY.
- **HYBRID OR MULTI-AGENT ENSEMBLES :** THESE REPRESENT AN ADVANCED FORM OF ENSEMBLE LEARNING, WHERE MULTIPLE DISTINCT LLM AGENTS COLLABORATE, INTERACT, OR COORDINATE THEIR PREDICTIONS TO HANDLE COMPLEX, MULTIFACETED TASKS. THE AGENTS INTERACT THROUGH A DEFINED COMMUNICATION MECHANISM , WHICH FACILITATES INFORMATION EXCHANGE AND COLLABORATION
- **MARS (MULTI-AGENT REVIEW SYSTEM):** A ROLE-BASED COLLABORATION FRAMEWORK INSPIRED BY THE REVIEW PROCESS. IN MARS, AN AUTHOR AGENT GENERATES AN INITIAL SOLUTION, REVIEWER AGENTS PROVIDE DECISIONS AND COMMENTS INDEPENDENTLY, AND A META-REVIEWER INTEGRATES THE FEEDBACK TO MAKE THE FINAL DECISION AND GUIDE FURTHER REVISION.

citations:

- MARS paper: <https://arxiv.org/abs/2509.20502>
- geeksforgeeks: <https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning/>
- youtube links: <https://www.youtube.com/watch?v=LsPi2wPZft8>
- <https://www.youtube.com/watch?v=sN5ZcJLDMaE>
- <https://medium.com/@duygujones/ensemble-learning-combining-models-for-better-machine-learning-d75d7be66b10>
- <https://towardsdatascience.com/ensemble-learning-stacking-blending-voting-b37737c4f483/>
- Ensemble large language models: A survey <https://www.mdpi.com/2078-2489/16/8/688>
- A comprehensive review on ensemble deep learning: Opportunities and challenges: https://www.sciencedirect.com/science/article/pii/S1319157823000228?ref=pdf_download&fr=RR-2&rr=9a343902da0e18a7



THANK YOU